

Amortizing Subgroup Discovery: A Precompute/Infer Architecture for Multi-Query Workloads

Matyáš Veselý¹[0009-0006-7994-4494], Viktor Brada², and Jiří Franc¹[0000-0002-5761-8675]

¹ Department of Mathematics, Faculty of Nuclear Sciences and Physical Engineering,
Czech Technical University in Prague, Czech Republic
`veselm73@fjfi.cvut.cz`, `jiri.franc@fjfi.cvut.cz`

² Workday, Prague, Czech Republic

Abstract. Subgroup discovery (SD) returns a handful of interpretable conjunctive rules describing where a target deviates from its base rate, yet it is almost always cast as a one-shot batch computation over a fixed table. A growing class of deployments breaks this assumption: a single base table is explored through many *views*, such as per-region or per-tenant dashboards, per-cohort filters, or role-based access-controlled (RBAC) slices, and each viewer needs discovery faithful to the rows they can actually see. Re-running a full SD pipeline per view is too slow to be interactive, and when scopes are rescoped at runtime, a rule set returned minutes later may describe a view its recipient can no longer see. We recast SD as a two-phase process and instantiate it as SDUEBA (Subgroup Discovery Using Embedding-Based Analysis). A one-time *precompute* stage learns row embeddings and clusters the base dataset, capturing latent structure shared across views; a lightweight *infer* stage refits short conjunctive descriptions on each visible slice. On a 2M-row, 45-state dashboard, a single ~ 20 -minute precompute is amortized across all states, after which SDUEBA answers each per-state query in about one second (mean 0.99 s, max 9 s on the largest state); the full 45-state sweep completes in ~ 21 minutes against ~ 66 for pysubgroup-beam rerun per state, while a single SSD++ pass over the 2M table already takes ~ 32 hours. Because views are read off a shared partition, the rule sets are near-disjoint by construction ($\sim 4\%$ overlap versus $\sim 99\%$ for top- k miners) and compact (≤ 2 literals). SDUEBA makes an explicit runtime-coverage tradeoff: it describes less of each table than SSD++, but returns its rules in seconds where a single from-scratch SSD++ pass is infeasible. Across 7 public datasets we characterize this tradeoff and the objective each SD family serves, so practitioners can match method to goal.

Keywords: subgroup discovery · knowledge discovery · embedding-based clustering · multi-query workloads · role-based access control · precompute/infer architecture.

Table 1: Deployment requirements for multi-query subgroup discovery.

Req.	Name	Implication
R1	Subgroup-list output	Return a compact, non-redundant rule set: few rules with minimal mutual overlap.
R2	Million-row scale	Ingest base datasets beyond 10^6 rows on commodity hardware.
R3	Seconds-scale latency	Answer each filtered query in seconds rather than rerunning a full SD pipeline.
R4	Short descriptions	Cap rule descriptions at a small constant to retain actionability in applied settings.

1 Introduction

Subgroup discovery across many views of one table. Subgroup discovery (SD) is a long-standing tool for knowledge discovery: from a labelled table it extracts a handful of interpretable conjunctive rules describing where a target deviates from its base rate [6,20,1]. It is almost always run as a one-shot batch task over a fixed dataset. A growing class of deployments breaks that assumption: a single base dataset D is explored through many *views*, where different users see different row subsets while sharing the same schema and the same underlying data-generating process. Per-region or per-tenant analytics, per-user cohort dashboards, and role-based access-controlled (RBAC) views all instantiate sparse visibility filters $f_q : D \rightarrow D_q \subset D$; we treat each such view as a *query* against D , and each viewer expects discovery *faithful to the rows they can actually see*. The question is then not only which subgroup objective to optimise, but which parts of discovery can be shared across many related views without making the rules returned for any one view unfaithful to it, and whether per-view discovery can be made fast enough to be interactive: after selecting a view, an analyst should see its rules within seconds.

These four requirements (Table 1) are coupled, and R3 is a hard bound rather than a preference. In the strictest deployments scopes evolve at runtime (e.g. RBAC grants are revoked or edited), so a pipeline whose per-query latency exceeds that update window can deliver results under a scope its recipient no longer holds. There, latency is a matter of correctness, not only of interactive comfort. In the more common case it simply bounds interactivity (§2.2).

Existing methods satisfy only parts of this joint specification. Search-based top- k miners (PYSUBGROUP, SUBGROUPS-BSD, SD-Map, VLSD [10,2,13]) return overlapping patterns and must be rerun whenever the visible rows change, violating R1, R2, and R3; subgroup-*set* miners (SSD++ and DSSD, Diverse Subgroup Set Discovery [17,8]) optimize a compact set under a redundancy penalty (R1) but are likewise recomputed from scratch on each D_q , violating R2 and R3. SSD++ alone needs ~ 32 h per 2M-row pass (§4.3), so running a fresh whole-table pass to serve each of many views is wholly impractical. §2.3 analyses why a precompute step cannot make either family’s per-query output cheap.

The common limitation is structural: prior methods compute their output as a function of the currently visible row set, so the discovery cost remains $\mathcal{O}(T_{\text{full}}(|D_q|))$ even when many views access the same base table. SDUEBA (Subgroup Discovery Using Embedding-Based Analysis) instead decomposes SD into dataset-level *structure discovery*, paid once, and per-view *description discovery*, linear in $|D_q|$. The evaluation in §4 reports results against R1–R4.

This paper makes three contributions. **(1)** We frame *multi-query* subgroup discovery: one base table explored through many sparse, runtime-evolving visibility filters (per-region, per-tenant, or per-cohort dashboards and RBAC slices), each requiring a compact rule set faithful to its visible rows. Unlike incremental or streaming SD, which revises rules as the *data* changes, this setting fixes the base and varies the *view*; to the best of our knowledge the SD literature has not treated it as a first-class problem. **(2)** We introduce SDUEBA, an embedding-based architecture that makes per-view discovery interactive by separating it into a one-time *precompute* stage (row embeddings and a clustering of D that capture dataset-level latent structure shared across all views) and a per-view *infer* stage that rediscovers short conjunctive descriptions and visibility-dependent scores on each slice D_q at second-scale latency. **(3)** In 7 public datasets, including a 2M-row dashboard workload, we characterize the *runtime-coverage tradeoff* SDUEBA makes against SSD++: SDUEBA describes less of each table, but returns its compact rule set in seconds on inputs where a single from-scratch SSD++ pass is infeasible (~ 32 h at 2M rows), and the precompute/infer split amortizes structure discovery across queries. We map which SD objective fits which deployment.

2 Background and Problem Setting

2.1 Subgroup discovery

Given a dataset D with target column T and target value t , a subgroup discovery method returns a set $\mathcal{R}$ of conjunctive rules; each $r_i \in \mathcal{R}$ covers a subset $S_i \subseteq D$ where the conditional probability $p_i = P(T=t \mid S_i)$ diverges from the base rate $p = P(T=t)$ [6,20,1]. Following [17], we distinguish *top- k subgroup miners*, which rank rules by individual quality, from *subgroup-set miners*, which optimise a set-level objective.

Top- k subgroup miners. Traditional SD scores each candidate rule by an individual quality function ϕ (typically weighted relative accuracy, WRAcc [7]) and returns the k highest-scoring rules. This paradigm has two recognised drawbacks [17]: quality functions track only the *centrality* of the target (mean, frequency), ignoring dispersion; and *pattern explosion* fills the top- k list with minor variants of one underlying rule [8]. Methods divide by search strategy. *Exhaustive miners* (SD-Map [2], the bitset BSD family [11,9], VLSD [13]) return the globally optimal top- k via optimistic-estimate pruning but cost exponentially in the worst case: in our setting, SD-Map and VLSD are infeasible above $\sim 5 \times 10^4$ -row datasets (§4.4)

and, as top- k miners, inherit the high-redundancy rule sets of the rest of their family, while the bitset BSD family, though the most scalable of the three, still hits memory or runtime walls beyond 10^5 rows at its default support threshold (§4.3). *Heuristic miners* trade completeness via beam search [7]; PYSUBGROUP’s beam, DFS, and APRIORI-SD [10,9] scale to $\geq 10^5$ rows without the optimality guarantee.

Subgroup-set miners (SSD) Subgroup *set* discovery targets redundancy directly, returning subgroups that jointly cover different regions with minimum mutual overlap [8,17]: by *sequential covering* (CN2-SD [7]), *diverse beam selection* under an overlap penalty (DSSD [8]), or *minimum-description-length (MDL) rule lists* (SSD++ [17]) that minimise the code length $L(\mathcal{R}) + L(D \mid \mathcal{R})$.

Benchmark setting We benchmark against the most scalable representative of each family: PYSUBGROUP-beam as the representative heuristic top- k miner, SUBGROUPS-BSD as the most scalable exhaustive top- k miner (SD-Map and VLSD are infeasible above $\sim 5 \times 10^4$ rows in our setting and, like the rest of the top- k family, return high-redundancy sets; see §4.4), and SSD++ as the representative MDL subgroup-set miner. Throughout the evaluation, SSD++ is the principal point of comparison for SDUEBA: both target the subgroup-*set* discovery setting (non-overlapping, compact rule sets rather than ranked top- k lists).

Quality measures. Unlike classification, SD has no single agreed-upon quality function: each method optimises its own objective, and the choice of measure is itself a modelling decision [1]. We therefore report several complementary measures, defined here over a rule set $\{r_i\}$ on a dataset D of N rows. Write S_i for the rows rule r_i covers, $p_i = p(T=t \mid S_i)$ for its target rate and p for the base rate on D , with Q_i and P the corresponding target distributions. Two of the measures use a *disjoint* version of the covers: ordering the rules best-first and attributing each row to the first rule that covers it yields blocks $S_i^* = S_i \setminus \bigcup_{j < i} S_j$ that partition D (the *sequential cover*) with Q_i^* the target distribution on block S_i^* ; we motivate this choice below. *Weighted relative accuracy* $\text{WRAcc}_i = (|S_i|/N)(p_i - p)$ is the per-rule, support-weighted precision lift introduced for CN2-SD [7] and used by the top- k miners PYSUBGROUP, the `subgroups` library, and the BSD family [10,11]. As a per-rule score it accounts for neither the overlap between subgroups nor their description length, so we define it for completeness but do not use it as a comparison metric below. At the set level, *coverage* $|\bigcup_i S_i|/N$ is the fraction of the table that some rule describes and *overlap* $1 - |\bigcup_i S_i| / \sum_i |S_i|$ is the fraction of rule mass that re-describes already-covered rows. *Sum-weighted KL divergence* $\text{SWKL} = \sum_i (|S_i^*|/N) \text{KL}(Q_i^* \parallel P)$ is the set-level divergence into which SSD++’s MDL code length $L(\mathcal{R}) + L(D \mid \mathcal{R})$ decomposes [17], evaluated over the sequential cover. Other set miners optimise yet other objectives: DSSD a description-overlap penalty [8].

Why the sequential cover. Scoring each instance once makes SWKL comparable across methods: over the raw (overlapping) covers, a high-overlap miner re-counts

the same divergence across near-duplicate rules and inflates the sum. With overlap ranging from $\sim 4\%$ (SDUEBA) and 0% (SSD++) to $97\text{--}99.6\%$ (beam, bsd; §4.4), the sequential cover is what compares them fairly.

2.2 The multi-query workload

We consider a fixed base dataset D of N rows together with a stream of K queries $q_1, \dots, q_K$. Each query carries a visibility filter $f_q : D \rightarrow D_q \subset D$ that restricts the rows visible to that query; filters are sparse selectors (an RBAC role, a US state, a user cohort), and the schema is shared across all D_q . The goal is to return an interpretable subgroup result $\mathcal{R}_q$ on each D_q at low *per-query* latency.

2.3 Why existing SD methods cannot decouple search from the query

The natural counterfactual is retrofitting a precompute step onto an existing method. Each family can amortize its data-structure substrate (cached supports, per-item coverage bitsets) but not its search: the objective that drives enumeration, pruning, and greedy construction is itself a function of the active population D_q . For *search-based miners* [10,9] the quality $\phi(r; D_q) = \text{WRAcc}$ depends on D_q through both the cover fraction and the base rate $p_q = P(T=t \mid D_q)$, so the top- k ranking shifts with D_q . Heuristic beam miners keep the top- b_w candidates by this D_q -dependent quality (optimistic-estimate pruning [9] compounds it), so a beam mined on D need not even *contain* the top- k for D_q . For *MDL subgroup-set miners* [17] the code length, its model-complexity penalty, and the sequential cover are all functions of D_q , so the list optimal under D is in general not optimal under D_q , and the greedy separate-and-conquer construction reruns per query. The substrate amortizes; the search, and hence the returned result, does not. SDUEBA, by contrast, makes its heavy stage (embedding and clustering) D_q -invariant, leaving only an $O(|D_q| \cdot \text{literals})$ refit per query.

3 The SDUEBA Method

SDUEBA represents rows in a composite embedding space, clusters them, and extracts one rule per cluster. The pipeline naturally separates into dataset-level structure (query-invariant) and per-query rule induction and metric scoring, which enables the precompute/infer split formalised in §3.3.

Why an embedding space. Casting rows into an embedding space before searching for rules makes the dataset-level structure reusable across views. Classical SD searches the raw conjunctive space directly, so each query re-pays the full search; SDUEBA instead folds the heterogeneous columns (categorical, numerical, and binary) into one continuous geometry whose proximity reflects how often feature values co-occur, then clusters it into coherent regions that each admit a short description. Crucially this geometry is a function of D alone: the query-invariant

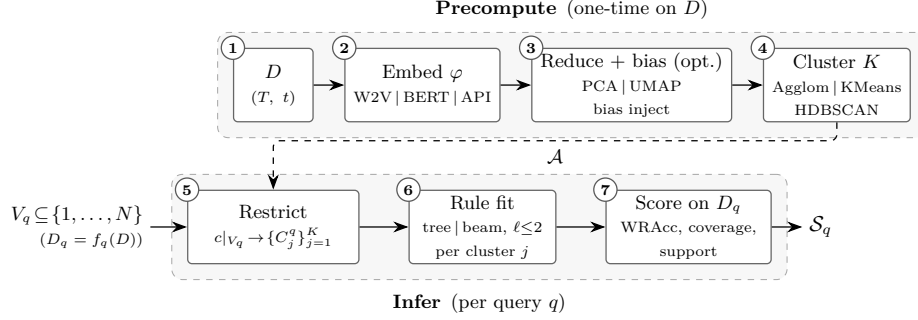


Fig. 1: SDUEBA pipeline; each circled number keys to the matching step in §3. **Precompute** (once on D): embed, optionally reduce/bias, cluster, and cache $\mathcal{A} = \langle \varphi, E, c \rangle$. **Infer** (per query q , given V_q): restrict c to V_q , fit a depth- ≤ 2 rule for each cluster with $|C_j^q| \geq n_{\min}$ and keep it if its accuracy $\geq \tau$, then score on D_q to give $\mathcal{S}_q$.

artifact that *precompute* caches once and every *infer* call reads off (§3.3). The categorical block is a swappable encoder, so the right encoding is a property of the data, not a commitment of the method. The benefit of the learned geometry is *targeted, not universal*: word2vec’s co-occurrence geometry is decisive on categorical-rich tables (mushroom, adult), whereas on near-Boolean or all-numeric data a one-hot baseline is competitive or stronger. The two coincide when there is no categorical block. SDUEBA’s mixed-type pipeline accommodates either through the same precompute/infer machinery. Table 2 reports the SWKL side of this ablation; cluster recoverability (on which one-hot matches or exceeds word2vec on the near-Boolean tables) and the raw/PCA-reduced one-hot breakdown are in the supplement.

Figure 1 numbers the seven pipeline steps; we walk through each below, the bold marker **(N)** keying the text to the matching box.

(1) *Input*. SDUEBA consumes a single table $D = (T, t)$: a feature matrix T over mixed-type (categorical, numerical, binary) columns and a binary target t flagging the property of interest. No train/test split or per-column preprocessing is required; every later step reads off this one object.

3.1 Composite row embedding

(2) **Embed φ** . The first stage produces one vector per row that mixes categorical semantics with numerical ranges: the per-type encodings described below are concatenated into a common vector space, so categorical, continuous, and binary features are handled in one pass from the raw table without a train/test split or manual per-column preprocessing. Categorical columns are tokenized as **feature=value** strings and a shared word2vec model [14] is trained over the row corpus; word2vec is the default rather than a pretrained transformer because the

vocabulary (e.g. `odor=pungent`) is dataset-specific and out-of-distribution for any general-purpose model, so in-corpus training yields a geometry discriminative for the values actually present. Pretrained-transformer and API backends are drop-in replacements for text-rich domains. Numerical and binary columns are not tokenized: numerical columns are standardised to zero mean and unit variance and binary columns are kept as 0/1 indicators, and these numeric blocks are concatenated with the categorical word2vec embedding to form the row vector. The output is $e(x_i) \in \mathbb{R}^d$ per row; default $d = 150$.

(3) Reduce + bias (optional). Before clustering, E can pass through the two transforms that most affect the downstream clustering, the *Reduce + bias* stage of Fig. 1. *Dimensionality reduction* compresses the embedding (PCA, default $d_{\text{pca}}=5$ at $N \geq 50\text{K}$, retaining 80–105 % of SWKL), which speeds up clustering at scale and drops noise directions that blur cluster boundaries. A *supervised bias term* (`BiasCfg`) then nudges the embedding toward target-class separation, so the clustering (and hence the candidate subgroups) aligns with the property of interest. Both are off by default (optional embedding normalisation is also available); their effect on quality is in §4.5.

3.2 Clustering

(4) Cluster K . The N (preprocessed) row vectors are clustered into K clusters using one of three backends: agglomerative clustering (default; K given a priori) [16], HDBSCAN (auto- K) [3], or spherical k -means [4]. The clustering output is an assignment $c : \{1, \dots, N\} \rightarrow \{1, \dots, K\}$, and each cluster becomes a candidate subgroup. For the agglomerative and k -means backends, K must either be tuned via grid search or estimated from the data. Spherical k -means is our recommended default at scale: it is the only backend whose precompute time grows linearly in N ($\alpha \approx 1.03$, against $\alpha \approx 1.5$ and $30\text{--}65\times$ overhead for HDBSCAN; §4.2), at a quality penalty against agglomerative clustering on smaller datasets (§4.5). The embedding map φ , the row embeddings E , and the assignment c are cached as the precompute artifact $\mathcal{A} = \langle \varphi, E, c \rangle$ (the dashed handoff in Fig. 1); every infer call reads $\mathcal{A}$ rather than recomputing it.

3.3 Infer: restrict, fit, and score

The precompute/infer split follows the divide between query-invariant and query-dependent work: the embeddings $\{e(x_i)\}$ and assignment c depend only on D , whereas the per-cluster rules $\{r_j^q\}$ and their scores depend on the query. Fitting rules per query is deliberate: the conjunction that best separates a cluster within a visible slice need not be the one that separates it on the full base, so two users seeing different slices get explanations grounded in their own rows. The partition c itself, however, is fixed during precompute: SDUEBA returns the globally coherent cohorts intersected with each slice, so a cohort that exists *only* within a particular D_q and crosses global cluster boundaries is by design out of scope, the price of amortizing structure discovery (revisited in §5.1).

(5) Restrict. A query q comes with a visibility set $V_q \subseteq \{1, \dots, N\}$ (the visible slice $D_q = f_q(D)$). Infer restricts the cached assignment to these rows, $c|_{V_q}$, giving per-query cohorts $\{C_j^q\}_{j=1}^K$. Cohorts smaller than $n_{\min}$ are dropped (the $|C_j^q| \geq n_{\min}$ gate in Fig. 1), so rules are fit only where the slice keeps enough support.

(6) Rule fit. For each surviving cohort, SDUEBA fits a binary classifier separating “in C_j^q ” from “not in C_j^q ” on the original feature columns, then reads a conjunctive rule off it. The default backend is a decision tree, whose root-to-leaf paths are conjunctions; its depth bound enforces requirement R4 (§1) structurally rather than as a tunable knob. A bounded beam-search backend is also available, and `allow_negations=False` yields negation-free descriptions. Only cohorts with classifier accuracy above τ are kept (the $\text{acc} \geq \tau$ gate), one rule per cohort; when a cohort yields several candidate rules, SDUEBA keeps the one with the highest PIQ = $|p_i - p|/\ell_i$ (Precision Interpretability Quotient), its default intra-cluster tie-break (a selection heuristic, not a cross-method comparison metric).

(7) Score. Each kept rule is scored *on the visible slice* D_q (WRAcc, coverage, and support against D_q , not the full base), so the returned set $\mathcal{S}_q$ describes exactly the rows the viewer can see.

Incremental updates. $\mathcal{A}$ admits append-only updates without re-running precompute: a new batch is embedded by the existing φ and assigned to nearest centroids by K-NN, matching the daily-append pattern in which D grows while the cluster geometry stays stable. The cost model and a concept-drift sentinel (future work, §5.2) are in the supplement (§S1).

4 Experimental Evaluation

This section evaluates SDUEBA on three axes, each in its own subsection: per-query *latency* (§4.3), *scaling* as N grows (§4.2), and per-rule and set-level *quality* (§4.4). Three findings anchor it. **(i) Latency:** after a one-time precompute, each query is a single *infer* call — 0.99 ± 1.59 s per state on the 2M-row 45-state dashboard, and 16–89 s for a whole-table 1M–5M query (vs. 50,201 s for SSD++ at 1M). **(ii) Scaling:** infer is near-linear (16–18 μ s/row), so precompute amortizes after a handful of queries. **(iii) Quality:** SDUEBA makes an explicit *runtime-coverage tradeoff* — it describes much less of each table than SSD++ (1–26% vs. 5–100% coverage) but returns its compact non-redundant set in seconds where a from-scratch SSD++ pass is infeasible; where coverage is comparable, its rules are as sharp per instance as SSD++’s (§4.4).

4.1 Setup

Datasets. The quality benchmark uses 6 binary-target datasets spanning 10^3 to $\sim 10^5$ rows — the moderate-to-large regime in which per-query subgroup discovery is deployed — drawn from UCI [5,18] and OpenML [19]; Table 2 reports the per-dataset characteristics. Scaling benchmarks use the US-Accidents table [15]

Table 2: Benchmark datasets: rows 1–6 for quality, row 7 (US-Accidents) for scaling. “Attr. (c/n/b)” is attribute counts by type (categorical/numeric/binary). Last column: sequential-cover SWKL with the categorical block encoded by word2vec vs. one-hot (**bold** = better; one-hot = stronger of its raw/PCA variants; §3); word2vec leads on categorical-rich tables, while all-numeric spambase coincides. The SWKL column for diabetes_130 is computed on a stratified 50K subsample (the cheap-ablation scope of §3). Recoverability and the raw/PCA breakdown are in the supplement.

#	Dataset	Instances	Attr. (c/n/b)	Target	Tgt. (%)	SWKL (w2v/1h)
1	mushroom	8,124	22 (16/0/6)	poisonous	48.2	0.69 / 0.40
2	adult	48,842	13 (7/5/1)	>50K	16.1	0.16 / 0.07
3	diabetes_130	101,766	45 (24/8/13)	readmitted	46.1	0.022 / 0.013
4	credit-g	1,000	20 (11/6/3)	good	70.0	0.025 / 0.040
5	kr-vs-kp	3,196	36 (1/0/35)	won	52.2	0.042 / 0.039
6	spambase	4,601	57 (0/57/0)	spam	39.4	0.217 (both)
7	US-Accidents	200K–5M	28 (5/6/17)	severe	19.4	—

at 200K–5M rows; the multi-query workload filters the 2M-row slice by **state** into 45 per-state queries.

Methods compared. On the quality benchmark, SDUEBA uses its library-default per-rule PIQ (Precision Interpretability Quotient) selector to keep one rule per cluster; all other hyperparameters (clustering family, K , embedding dimension, accuracy threshold) are tuned *per dataset*, exactly as every baseline is. To compare on equal footing, each method is tuned over its *full set of viable hyperparameters* at a matched top- $k = 10$ budget and reported at its per-(dataset, metric) tuned best; the number of configurations differs by method only because the methods are parameterised differently. Rule length follows each method’s standard operating point: SDUEBA keeps its library-default descriptions of at most two conditions (decision-tree depth 2, a structural choice rather than a tuned axis; §3), while the rule-search baselines are free to use the longer conjunctions their search favours. On the US-Accidents scaling runs (§4.2–§4.3) we report a single SDUEBA configuration.

Implementations and hardware. All experiments run on an academic HPC cluster (CentOS 7.8): ~ 25 multi-core x86 CPU nodes (128 GB RAM each) for the bulk, and two NVIDIA Tesla V100 (16 GB) GPU nodes for the transformer-embedding backend. We use our own SDUEBA reference implementation and the **rulelist** [17], **pysubgroup** [10], and **subgroups** [12] packages for the baselines. Each SDUEBA quality-benchmark cell runs 10 seeds; the deterministic baselines run once, and the scaling cells (§4.2) use 3 seeds. Absolute runtimes carry $\sim \pm 10\%$ hardware- and seed-level variance, so the speedups below are order-of-magnitude rather than exact. *Reproducibility:* versioned wheels, benchmark configs, merged

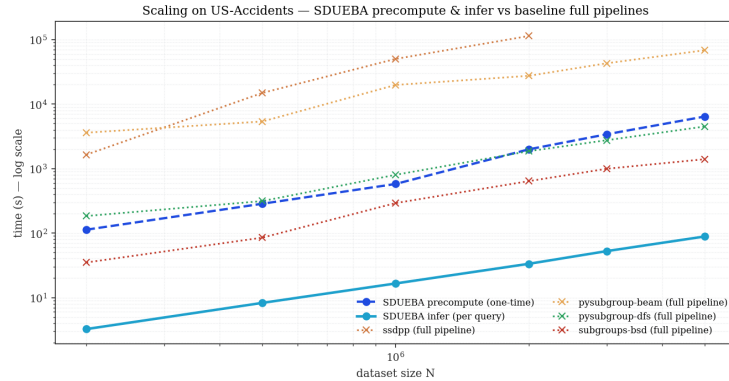


Fig. 2: Scaling on US-Accidents (log-log), representative `sdueba-k-means` configuration. SDUEBA’s one-time precompute and per-query `infer` share axes with the baselines’ full per-query runtime; `infer` sits one to three orders of magnitude below every baseline.

result CSVs, and per-run environment snapshots accompany the supplementary archive; the benchmark harness and SDUEBA library are public.³

Statistical methodology. Per-rule quality is compared per covered instance via the density $\rho = \text{SWKL}/\text{Cov}$ (the mean per-instance $\text{KL}(\theta_a \parallel \theta_d)$; §4.4). Because ρ averages divergence over only the covered rows, it is a fair per-instance comparison only between near-disjoint covers of *comparable size* — which holds here (mean residual overlap $\sim 4\%$ for SDUEBA and 0% for SSD++). Per-query latency is mean $\pm$ s.d. across 10 seeds; deterministic methods run once.

4.2 Scaling profile

Figure 4.2 overlays the precompute and per-query `infer` profiles on US-Accidents. Precompute time is dominated by the embedding and clustering stages and is independent of the rule-ranking selector (which acts only per query). Per-query `infer` time scales near-linearly in N and stabilises in the range $16\text{--}18\mu\text{s}$ per row from 200K to 5M.

4.3 Per-query speedup (latency)

Table 3 reports *single-query* latency — the cost of serving one user request over the full N -row table (serving one query per user is the dashboard below). We report two speedups, which answer different questions. *Amortization (self-)speedup* measures what the precompute/`infer` split saves over running SDUEBA

³ https://github.com/veselm73/sdueba_comparison, <https://github.com/veselm73/sdueba-lib>

Table 3: US-Accidents per-query timings vs. N (s); each query spans the full N -row table (SDUEBA: mean of 3 seeds). Bottom row: deployment requirements met ($\checkmark/\times$; glyphs ordered R1 R2 R3 R4 — R1 compact non-redundant set, R2 $\geq 10^6$ rows, R3 seconds latency, R4 short rules).

N	SDUEBA T_{pre}	SDUEBA T_{infer}	SSD++	PYSUBGROUP -beam	SUBGROUPS-BSD	Speedup vs SSD++
200K	113	3.2	1 644	3 603	35	507 $\times$
500K	285	8.3	14 920	5 378	85	1,803 $\times$
1M	577	16.5	50 201	19 808	292	3,035 $\times$
2M	1 988	33.4	113 849	27 453	639	3,411 $\times$
3M	3 405	52.5	—	42 957	993	—
5M	6 420	88.6	—	68 438	1 399	—
R1–R4	$\checkmark\checkmark\checkmark\checkmark$		$\checkmark\ \times\ \times\ \times$	$\times\ \times\ \times\ \times$	$\times\ \times\ \times\ \times$	

from scratch each query: the one-time T_{pre} dominates the from-scratch total, so a single **infer** call costs only a small fraction of a full (“naive”) SDUEBA run that redoes embedding and clustering, a saving of roughly 37–75 $\times$ across the tested scales. *Cross-method latency* is the deployment-relevant comparison (R3): SDUEBA-infer answers each query in seconds where SSD++-tuned and PYSUBGROUP-beam are orders of magnitude slower (Table 3), a gap that widens with N . Only SUBGROUPS-BSD, run at a relaxed `min_support` (without which bitset enumeration is infeasible beyond $\sim 10^5$ rows), stays competitive on raw latency, but its top- k output is almost entirely redundant (§4.4) where SDUEBA’s is near-disjoint, so the comparison is not iso-quality. The one-time precompute is therefore amortized almost immediately: at every $N \geq 200\text{K}$ a single filtered query already costs less than one from-scratch run of SSD++ or PYSUBGROUP-beam, and only a few queries suffice to overtake even the much cheaper but quality-degraded SUBGROUPS-BSD.

The 45-state dashboard. The deployment-realistic case is the multi-query workload of §2.2: each query is one state’s slice of the 2M table. One ~ 20 min precompute (this dashboard’s own configuration and run) is shared across all states; each per-state **infer** then costs 0.99 ± 1.59 s (range 0.03–8.96 s; California, 403K rows, is the slowest). The full 45-state sweep completes in ~ 21 min against ~ 66 min to rerun PYSUBGROUP-beam per state, and each extra state or refresh adds only ~ 1 s rather than a full re-mine — per-query latency, not the one-time precompute, is what deployment must meet.

4.4 Quality: an explicit runtime–coverage tradeoff

On the two set-level metrics — coverage and SWKL — SSD++ leads, for a clear reason: its MDL objective maximises exactly the sequential-cover SWKL (§2.1), while SDUEBA, by design, describes only a non-redundant fraction of each

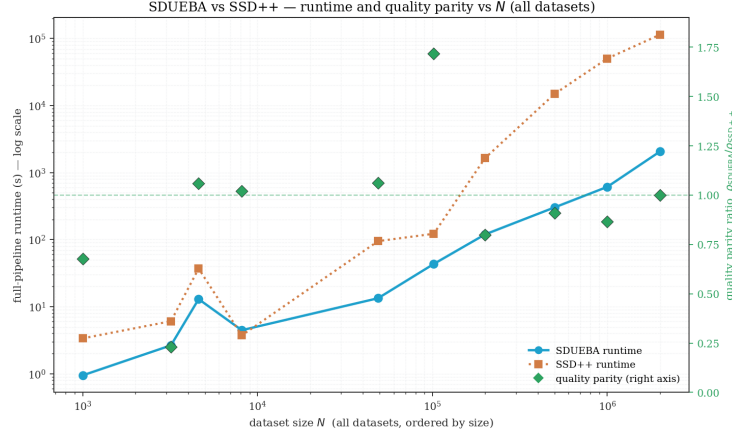


Fig. 3: SDUEBA vs. SSD++, datasets ordered by N . *Left axis (lines)*: full-pipeline runtime — SDUEBA below SSD++ everywhere, gap widening with N . *Right axis (diamonds)*: per-instance divergence parity $\rho_{\text{SDUEBA}}/\rho_{\text{SSD++}}$ about the 1.0 line. Where the two cover comparable fractions, SDUEBA’s per-instance divergence is on par with SSD++’s at a fraction of the runtime; SSD++’s SWKL lead is driven by its higher coverage.

table. To separate the coverage increment from the quality increment in SWKL, we measure the per-instance *density* $\rho = \text{SWKL}/\text{Cov}$: the average divergence of a single covered instance — how sharp each described row is, independent of how many rows are described — formally the mean $\text{KL}(\theta_a \parallel \theta_d)$ over covered instances, each counted once.

Figure 3 overlays full-pipeline runtime (lines) and the parity ratio $\rho_{\text{SDUEBA}}/\rho_{\text{SSD++}}$ (points) against N . The dominant fact is the *coverage* gap: SDUEBA describes 1–26% of each table against SSD++’s 5–100%, and SSD++’s set-level lead on coverage and SWKL follows directly. Within the slice each method does describe, per-instance density is comparable: the ratio sits near 1 on the larger categorical-rich tables (spambase and adult near 1.06, mushroom 1.02; US-Accidents climbing $0.80 \rightarrow 1.00$ from 200K to 2M), so SDUEBA’s described rows are individually about as divergent from the base rate as SSD++’s. The exceptions are the two smallest datasets, credit-g (0.68) and kr-vs-kp (0.23), where SSD++’s much denser cover (91% on kr-vs-kp) is sharper per instance than SDUEBA’s disjoint clusters — the honest limit of a partition-first design, where the high-divergence subgroups are feature conjunctions a beam isolates but no row clustering recovers. The tradeoff is thus less of each table described in exchange for a compact, non-redundant set of equally sharp cohorts delivered in seconds — the deciding factor at the scales of Table 3, where a from-scratch SSD++ pass is infeasible and its higher coverage is unavailable at any tractable cost.

The structural overlap of SDUEBA’s rule set is $\sim 4\%$ — a property of the partition (clusters disjoint by construction), not a tuning artifact — against PYSUBGROUP-beam’s $\sim 99.6\%$ and SUBGROUPS-BSD’s $\sim 97\%$, while SSD++ is 0% by sequential-cover construction. Of the deployment requirements R1–R4 (§1), only SDUEBA satisfies all four; the obstacles for each baseline family are analysed in §2.3.

4.5 Sensitivity and stochasticity

Stochastic variance across 10 seeds is 10–33% and is largest on small datasets; all other methods in the comparison are deterministic given fixed data and hyperparameters (verified over multiple seeds and hash orders). Failure modes are predictable: at $N \geq 2\text{M}$ the embedding tensor exceeds 16 GB RAM (32 GB recommended), and at $N < 300$ rows with high K the pipeline returns zero subgroups. The per-axis marginal-effect plots are included in the supplement.

5 Discussion and Conclusion

5.1 Limitations

SDUEBA exposes a large grid over embedding, clustering, K , dimension, τ , rule backend, and negation, so *finding* a good configuration is itself work, and no universal default exists (§4.5). SSD++, by contrast, reaches a strong SWKL operating point with no dataset-specific tuning, a usability advantage the headline speedup does not capture. On latency the advantage flips, and tuning cannot close the gap: a set-miner re-searches on every query, so its per-query cost is structural rather than a matter of configuration. Fixing the partition during precompute means SDUEBA can only clip its global clusters to each view, so a cohort that coheres *only* within one view’s rows, cutting across those global clusters, cannot be recovered (§3.3). This holds only while the global partition stays appropriate across views; the assumption is borne out here (stable per-state quality on the 45-state dashboard) but can fail under extreme visibility skew or concept drift. We also do not benchmark a cheaper substitute for a full SSD++ pass: running it on a uniform random *sample* rather than the whole table. That would be fast and would likely recover much of SSD++’s coverage, but the cost cannot be shared across views: each view must draw and re-mine its own sample, whereas SDUEBA embeds and clusters once and reuses that work for every view. Finally, our non-overlap and “actionability” claims rest on metrics rather than a human study.

5.2 Future work

The precompute/infer split opens directions one-shot SD cannot reach. Because a single precompute serves *all* views, *cross-view* analysis becomes natural: tracking how a cohort’s target divergence shifts across RBAC roles or regions, or surfacing subgroups that appear under one filter and vanish under a tighter one. A

concept-drift sentinel that detects when the cached partition has degraded and re-clusters only the affected region would extend the amortization guarantee to non-stationary data. Because SDUEBA embeds rows rather than discretising features, API-based LLM embeddings could also bring SD to tables with free-text *description* columns (clinical diagnoses, incident notes) that classical SD cannot reach. Finally, a learned configurator predicting a clustering family and K from cheap descriptors would cut the per-dataset tuning that remains SDUEBA’s main usability cost.

5.3 Conclusion

SDUEBA opens subgroup discovery to a deployment class one-shot SD has barely served: governed, multi-query analytics spanning the per-tenant, per-cohort, and RBAC views of §2.2 over one shared base table. Its embedding pipeline admits a natural precompute/infer split, and that split (one shared precompute, then sub-second per-query *infer*) is the right primitive for this workload and scales to tables a from-scratch miner cannot touch. The same split also updates *incrementally*: as the base table grows, new rows are embedded and assigned to existing clusters in $O(|\Delta|)$ with no re-precompute (§3), so a long-lived dashboard pays only for its new data.

The bargain is explicit: SDUEBA trades *coverage* (it describes less of each table than SSD++) for inference speed and scalability, while retaining a near-disjoint rule set ($\sim 4\%$ overlap). That trade is worth making in real-world analytics where, over a multi-query slice exceeding 1M instances, a short, actionable account of where the target diverges is more useful than a full-coverage explanation at infeasible compute cost (~ 32 h for a from-scratch SSD++ pass at 2M rows) or a large, redundant rule set that is hard to act on.

SDUEBA is the right tool when the workload is multi-query over shared base data, when low per-query latency is required, and when $N \geq 100K$; for single-query analyses on small datasets ($N < 1K$) the precompute setup is wasted overhead. The choice of method is best read by objective: PYSUBGROUP-beam for top- k WRAcc, SSD++ for the set-level metrics (coverage and SWKL) via MDL, and SDUEBA for short, sharp rules at near-zero overlap. We compute SWKL on the sequential cover, so the divergence-maximising method wins the divergence-maximising metric without re-counting overlapping rules. Where coverage is comparable, SDUEBA’s rules match SSD++’s per-instance divergence at near-zero overlap, and it answers per-query SD on million-row tables in seconds: roughly $37\times$ faster than a naive from-scratch run, and about a second per state on the 2M-row, 45-state dashboard.

References

1. Atzmueller, M.: Subgroup discovery – an advanced review. WIREs Data Mining and Knowledge Discovery **5**(1), 35–49 (2015)

2. Atzmueller, M., Puppe, F.: SD-Map – a fast algorithm for exhaustive subgroup discovery. In: Knowledge Discovery in Databases (PKDD). Lecture Notes in Computer Science, vol. 4213, pp. 6–17. Springer (2006)
3. Campello, R.J.G.B., Moulavi, D., Sander, J.: Density-based clustering based on hierarchical density estimates. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD). pp. 160–172. Springer (2013)
4. Dhillon, I.S., Modha, D.S.: Concept decompositions for large sparse text data using clustering. *Machine Learning* **42**, 143–175 (2001)
5. Kelly, M., Longjohn, R., Nottingham, K.: The UCI machine learning repository. <https://archive.ics.uci.edu>, university of California, Irvine
6. Klösgen, W.: Explora: a multipattern and multistrategy discovery assistant. In: Fayyad, U.M., Piatetsky-Shapiro, G., Smyth, P., Uthurusamy, R. (eds.) *Advances in Knowledge Discovery and Data Mining*, pp. 249–271. AAAI/MIT Press (1996)
7. Lavrač, N., Kavšek, B., Flach, P., Todorovski, L.: Subgroup discovery with CN2-SD. *Journal of Machine Learning Research* **5**, 153–188 (2004)
8. van Leeuwen, M., Knobbe, A.: Diverse subgroup set discovery. In: *Data Mining and Knowledge Discovery*. vol. 25, pp. 208–242 (2012)
9. Lemmerich, F.: Novel techniques for efficient and effective subgroup discovery. Ph.D. thesis, Universität Würzburg (2014), origin of the BSD bitset-based exhaustive search algorithm
10. Lemmerich, F., Becker, M.: pysubgroup: Easy-to-use subgroup discovery in Python. In: ECML PKDD (Demo Track). Lecture Notes in Computer Science, vol. 11053, pp. 658–662. Springer (2018)
11. Lemmerich, F., Rohlf, M., Atzmueller, M.: Fast discovery of relevant subgroup patterns. In: *Proceedings of the Twenty-Third International Florida Artificial Intelligence Research Society Conference (FLAIRS)* (2010)
12. López-Martínez-Carrasco, A., Juárez, J.M., Campos, M., Canovas-Segura, B.: subgroups: a Python library for subgroup discovery. <https://github.com/antoniolepezmc/subgroups> (2023), software package; verify peer-reviewed citation if available
13. López-Martínez-Carrasco, A., Juárez, J.M., Campos, M., Canovas-Segura, B.: VLSD—an efficient subgroup discovery algorithm based on equivalence classes and optimistic estimate. *Algorithms* **16**(6), 274 (2023)
14. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J.: Distributed representations of words and phrases and their compositionality. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2013)
15. Moosavi, S., Samavatian, M.H., Parthasarathy, S., Ramnath, R.: A countrywide traffic accident dataset (2019)
16. Müllner, D.: Modern hierarchical, agglomerative clustering algorithms (2011)
17. Proença, H.M., Grünwald, P., Bäck, T., van Leeuwen, M.: Robust subgroup discovery: discovering subgroup lists using MDL. *Data Mining and Knowledge Discovery* **36**, 1885–1970 (2022)
18. Strack, B., DeShazo, J.P., Gennings, C., Olmo, J.L., Ventura, S., Cios, K.J., Clore, J.N.: Impact of HbA1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records. *BioMed Research International* **2014**, 781670:1–781670:11 (2014)
19. Vanschoren, J., van Rijn, J.N., Bischl, B., Torgo, L.: OpenML: networked science in machine learning. *ACM SIGKDD Explorations Newsletter* **15**(2), 49–60 (2014)
20. Wrobel, S.: An algorithm for multi-relational discovery of subgroups. In: *Principles of Data Mining and Knowledge Discovery (PKDD)*. Lecture Notes in Computer Science, vol. 1263, pp. 78–87. Springer (1997)